- *The identity provider has a single responsibility*. The identity provider is responsible only for authentication and managing users. In many cases, this is generic enough that you can (and should!) use a third-party identity service, such as Auth0 or Azure Active Directory, instead of building your own.
- *You can easily add new sign-in mechanisms*. Whether you use the identity provider approach or the traditional approach, it's possible to use external services to handle the authentication of users. You'll have seen this in apps that allow you to "log in using Facebook" or "log in using Google," for example. If you use a centralized identity provider, you can add support for more providers in one place instead of having to configure every app and API explicitly.

Out of the box, ASP.NET Core supports architectures like this and for consuming bearer tokens from identity providers, but .NET 7 doesn't include support for issuing those tokens in the core framework. That means you'll need to use another library or service as the identity provider.

As I mentioned in chapter 23, one excellent option is to use a third-party identity provider, such as Facebook, Google, Okta, Auth0, or Azure Active Directory. These providers take care of storing user passwords, authenticating using modern standards like WebAuthn (https://webauthn.guide), and looking for malicious attempts to impersonate users.

By using an identity provider, you leave the tricky security details to the experts and can focus on the core purpose of your business, whichever domain that is. Not all providers are equal, though: For some providers (such as Auth0) you own the profiles, whereas for others (Facebook or Google) you don't. Make sure to choose a provider that matches your requirements.